

FlowForge Execution Engine, n8n Comparison & Complete Gap Analysis

1. Architecture Overview & Execution Mechanics

FlowForge (the n8n/Make.com replica) is structured as a decoupled monorepo composed of three main services:

- **Frontend (apps/web)**: A React + Vite + React Flow canvas application for visual workflow composition, node parameter editing, schema inspection, and real-time execution monitoring.
 - **Backend API (apps/api)**: An Express REST API that handles auth, workflow CRUD, webhook ingestion, trigger polling, credentials encryption, and execution enqueueing via BullMQ.
 - **Execution Worker (apps/worker)**: A Node.js worker service that consumes execution jobs from Redis/BullMQ, executes workflow graphs using an async topological DAG runner, resolves expressions per-item, and broadcasts real-time execution states via Socket.io / PubSub.
-

2. Deep Dive: The Execution Engine

The engine is primarily implemented in `apps/worker/src/engine/executor.ts` and supported by `expressions.ts`, `jsonPath.ts`, and `topoSort.ts`.

2.1 Level-Based Topological Execution (`computeLevels`)

1. **Annotation Stripping**: Non-executable nodes (`stickyNote`, `group`) are filtered out prior to execution planning (`stripAnnotationNodes`).
2. **Kahn's Topological Sort**: `computeLevels` calculates node in-degrees and groups executable node IDs into sequential execution levels (`string[][]`).
 - Nodes in the same level have zero dependencies on each other and run concurrently via `Promise.all()`.
 - Cycle detection ensures graph acyclicity. (Cyclic loops are strictly disallowed and must use `forEachBranch` instead).
3. **Branch & Skip Propagation**:
 - For branching nodes (e.g. `if`, `switch`), the plugin returns a `branch` identifier corresponding to the output handle.
 - Incoming edges to downstream nodes are evaluated: if all incoming source nodes failed, were skipped, or selected a different output handle, the downstream node is marked `skipped` and propagated without executing.

2.2 Item-Pairing Dataflow Model (`NodeItems`)

Data flows between nodes as an array of items (`NodeItems`), conforming to:

```
export interface NodeItem {  
  json: Record<string, unknown>;  
}
```

```
binary?: Record<string, BinaryData>;
pairedItem?: { item: number; sourceNode?: string };
}
```

- **Lineage Tracking:** `pairedItem` links derived outputs back to the input item index that produced them.
- **Backward Compatibility:** Legacy plugins returning `{ output: ... }` are auto-normalized into `NodeItems` by `normalizeToItems()`. When plugins expect single inputs, `itemsToLegacyValue()` extracts unwrapped objects.

2.3 Expression Resolution Engine (`resolveExpressions`)

Before executing a node plugin on a batch, expressions formatted as `{{ <code/path> }}` are resolved in `expressions.ts` per item.

- **Item Context:** Every item provides a scope containing:
 - `$json`: The current item's JSON payload.
 - `$item`: The full `NodeItem` (including `binary` and `pairedItem`).
 - `$node["Node Label"].json`: The representative JSON from the first item of a referenced upstream node.
 - `$node["node_id"].json`: Lookups by node ID.
 - `$vars`: Workflow-level global variables.
 - `$trigger`: The original trigger payload that initiated the workflow run.
 - `$workflow` / `$execution`: Workflow and execution metadata IDs.
 - `$getWorkflowStaticData()` / `$setWorkflowStaticData()`: Persistent workflow state across runs.
- **Expression Error Tracking:** Expression evaluation errors are captured per parameter and surfaced in execution telemetry without immediately crashing unhandled loops unless non-recovered.

2.4 Sub-Node Handle Wiring (`params.$subNodes`)

FlowForge supports diamond sub-node handles for AI and LLM building blocks:

- Handle types include: `ai_languageModel`, `ai_memory`, `ai_tool`, `ai_embedding`, `ai_textSplitter`, `ai_vectorStore`, `ai_outputParser`.
- During execution, incoming non-main edges are harvested by `executor.ts` and passed to the node plugin as `params.$subNodes.<handleId>`.
- Nodes like `agentNode` and `ragNode` inspect `params.$subNodes` to dynamically attach connected chat memories, vector stores, tools, or structured output parsers.

2.5 Control Flow & Advanced Execution Modes

- **Pause & Resume (`waitForWebhook`, `humanApproval`):**
 - Halts execution at the pause node.
 - Serializes graph state, outputs, and branch maps into a database checkpoint.
 - Generates a UUID `resumeToken` and updates the execution status to `paused`. Resumption is triggered via the `/resume` route.

- **Sub-workflows (`subWorkflow`):**
 - Recursively invokes `executeWorkflow` up to a max depth of 5 (`MAX_SUBWORKFLOW_DEPTH`).
- **Subgraphs (`forEachBranch`):**
 - Takes a nested subgraph definition in `params.subgraph` and runs loop iterations sequentially or in parallel over input array items.
- **Retries & Resilience:**
 - Per-node retry settings (`retry.maxAttempts`, `retry.delayMs`).
 - `continueOnFail`: Converts runtime exceptions into soft error payloads (`{ error: string, continuedOnFail: true }`).
 - `errorWorkflowId`: Automatically executes a configured fallback error workflow upon job failure.

3. Working Directory Changes (Your Active Session)

The current working directory contains several key additions and refinements:

3.1 Real Core Airtable Integration

- **File Created:** `apps/worker/src/nodes/airtableNode.ts`
- **Capabilities:** Full Web API integration with Airtable (List Records, Get Record, Create Record, Update Record, Upsert Record with match fields, and Delete Record).
- **Wiring:**
 - Registered in worker registry (`apps/worker/src/nodes/index.ts`).
 - Parameter UI schema added to `apps/web/src/lib/paramSchemas.ts`.
 - Node metadata added to `apps/web/src/lib/nodeTypeMeta.ts`.
 - Sample mock inputs updated in `apps/api/src/marketplace/registryIndex.ts`.

3.2 Dedicated Webhook Triggers for Calendly & DocuSign

- **Files Created / Modified:**
 - `apps/worker/src/nodes/triggerNodes.ts`: Added `calendlyTrigger` and `docusignTrigger` no-op trigger nodes.
 - `apps/api/src/utils/webhookSignature.ts`: Created signature verification helpers:
 - **Calendly**: Parses `t=...`, `v1=...` timestamp and HMAC-SHA256 signature headers.
 - **DocuSign Connect**: Computes base64 HMAC-SHA256 signatures for `X-DocuSign-Signature-1..5` headers.
 - `apps/api/src/index.ts`: Extended `express.json()` with `verify: (req, _res, buf) => { req.rawBody = buf; }` to capture raw unparsed body bytes necessary for cryptographic HMAC checks.
 - `apps/api/src/routes/webhook.ts`: Updated test and production webhook listener routes to auto-verify signature headers against the node's `signingSecret` before enqueueing execution jobs. Return `401 Unauthorized` on signature failure.
 - Shared Types & UI: Extended `ExecutionJobData['triggerType']`, added parameter schemas (`path`, `signingSecret`), icons, and palette meta.

3.3 Expression Errors UI Wiring

- **Files Modified:**

- `apps/web/src/components/NodeConfigPanel.tsx`: Added `lastRunExpressionErrors` prop and rendered an inline amber alert box displaying expression failures from the last run directly above parameter inputs.
- `apps/web/src/pages/CanvasPage.tsx`: Connected `selectedNode.data.lastRunExpressionErrors` to `NodeConfigPanel`.

4. FlowForge vs Official n8n Detailed Comparison

Domain	FlowForge / Replica	Official n8n
Graph Execution	Level-based topological DAG batch execution (<code>runLevels</code> using Kahn's algorithm).	Event-driven step-by-step queue execution via <code>WorkflowExecute</code> engine with node hooks.
Data Flow	Canonical <code>NodeItems</code> (<code>{ json, binary, pairedItem }</code>) with backward-compatible unwrapping.	<code>INodeExecutionData[]</code> array with binary metadata and explicit item-linking indexes.
AI Node Architecture	Sub-node handles (<code>\$subNodes</code>) drawn as diamond ports for direct canvas connections.	LangChain integration nodes using specialized sub-connections in <code>@n8n/n8n-nodes-langchain</code> .
Expression Engine	JS function-scoped evaluator with <code>\$json</code> , <code>\$node</code> , <code>\$vars</code> , <code>\$trigger</code> bindings.	Custom parser & sandbox built on isolated execution scopes (<code>vm2</code> / <code>isolated-vm</code>).
Dynamic Parameters	Static parameter schemas defined in <code>paramSchemas.ts</code> with custom expressions.	Server-backed dynamic methods (<code>loadOptions</code> , <code>resourceLocator</code> , dynamic search).
Community Ecosystem	Built-in node plugins in <code>apps/worker/src/nodes/</code> + simple community loader.	Full npm-based community node package distribution (<code>n8n-nodes-base</code>).

5. Complete Gap Analysis & Action Plan

5.1 Remaining Unbuilt Nodes

- **Opsgenie:** Opsgenie is the single remaining core node integration from the original target scope that has not yet been implemented.
 - *Action:* Create `apps/worker/src/nodes/opsgenieNode.ts` supporting alert creation, listing, closing, and updating via Opsgenie REST API.

5.2 Pending Dependencies & Verification

- **Uninstalled Worker Dependencies:** `ssh2-sftp-client` and `basic-ftp` were added to `apps/worker/package.json` for SFTP node capabilities, but need `pnpm install` in an

environment with network access.

- **Action:** Run `pnpm install` at root, followed by `pnpm -F worker exec tsc --noEmit` and `pnpm -F api exec tsc --noEmit`.

5.3 Engine Security & Sandboxing

- **Expression Isolation:** Currently, expressions are evaluated using `new Function()`.
 - **Action:** In a production environment, wrap expression resolution in a sandboxed isolate (e.g. `isolated-vm` or Node `vm` module with strict global scope isolation).

5.4 Canvas Pre-Flight Validation

- **Required Field Validation Badging:** `required: true` properties exist in `paramSchemas.ts` and `nodeIssues.ts` helper, but visually highlighting red issue badges on canvas nodes prior to run needs complete UI pass across all ~150 node types.

5.5 Control Flow Limitations

- **Nested Pause Limitation:** Pausing execution (`waitForWebhook`, `humanApproval`) inside nested `subWorkflow` calls or `forEachBranch` subgraphs is currently blocked by design.
 - **Action:** Implement bubble-up checkpointing for nested sub-workflow state if async human approval inside loops/sub-workflows is required.

5.6 Platform & Infrastructure Gaps

- **Enterprise SSO:** `SsoSettingsPage.tsx` and `RbacPage.tsx` components exist in the frontend, but backend SAML/OIDC authentication protocols need complete verification against external identity providers (e.g. Okta, Azure AD).
- **Audit Logging Completeness:** `ActivityLog` rows are generated for primary actions, but exhaustive logging across all sensitive mutations (credential creation/deletion, workspace role updates, marketplace installs) requires full audit verification.
- **Notification Channel Delivery:** Execution alert dispatching currently supports in-app logging; external delivery channels (Slack webhook triggers, SMTP email alerts) must be verified against live endpoints.
- **API Rate Limiting Enforcement:** Middleware rate limits (`RATE_LIMIT_PER_MINUTE`) must be strictly applied across all public ingress points (`/webhook/*`, `/chat/*`, `/auth/*`).
- **Data Retention Background Cron:** The background job enforcing `EXECUTION_RETENTION_DAYS` must run periodically to clean up historical execution runs and binary storage buffers.
- **Canvas Mobile Responsiveness:** Canvas drag-and-drop node placement and connection edge drawing are optimized for desktop viewports; mobile touch interactions require enhanced touch-event handlers.
- **Real-Time Canvas Collaboration:** Workflow comments and version histories exist; full Google Docs-style real-time multi-cursor collaboration on the canvas is not implemented.

5.7 API & Realtime Socket Handling

- **Realtime Broadcast Rooms:** Socket events now route through `broadcastRooms()` (`io.to(['user:${ownerId}', 'workflow:${id}'])`), ensuring collaborators on the canvas

receive live node states simultaneously without duplicate events.

- **Chat & Webhook Test Paths:** `/chat/test/:workflowId/:path` and `/webhook/test/:workflowId/:path` allow draft/unpublished workflows to be tested live without requiring full workflow activation.

5.8 Binary Data & Attachment Storage Architecture

- **In-Memory Base64 Buffer:** FlowForge processes attachments using in-memory base64 buffers (`makeBinary` / `decodeBinary`). For multi-gigabyte files or high-concurrency file processing, an offloaded binary storage driver (S3 / local disk filesystem stream pointers, similar to n8n binary data mode) would optimize worker RAM consumption.
- **Binary Previews Cap:** Executor strips base64 payloads over 512KB (`BINARY_PREVIEW_MAX_BYTES`) for frontend UI previews to maintain websocket responsiveness.

5.9 OAuth2 Token Lifecycle & Credential Management

- **Centralized OAuth Refresh:** `oauthRefresh.ts` implements generic token refreshes 60s prior to token expiry across all `oauth2` credentials without per-integration code duplication.
- **Provider Env Config:** OAuth refresh depends on provider-specific client ID / secret environment variables (`OAUTH_TOKEN_PROVIDERS`); missing env settings fallback gracefully to standard 401 response handling.

5.10 Native Trigger Polling Connections

- **Native Protocol Consumers:** Kafka, RabbitMQ, Redis Streams, PostgreSQL LISTEN/NOTIFY, IMAP IDLE, and FS file watchers run native persistent background connections via `triggerPollers.ts` and `emailPoller.ts`.
- **Scaling Multiple Instances:** When running multiple API/worker instances, Kafka and RabbitMQ consumer groups prevent duplicate message execution, while PostgreSQL LISTEN/NOTIFY connections should be scoped per active workflow.

6. Priority Action Checklist

1. ☐ **Dependencies & Types:** Run `pnpm install` and verify types with `pnpm -F worker exec tsc --noEmit` and `pnpm -F api exec tsc --noEmit`.
2. ☐ **Opsgenie Integration:** Build `apps/worker/src/nodes/opsgenieNode.ts` to close the final missing integration gap.
3. ☐ **Sandbox Security:** Wrap `resolveExpressions` in an isolated execution sandbox (`isolated-vm`).
4. ☐ **Pre-Flight Canvas Badges:** Finalize visual indicator badges on canvas nodes for required missing fields across all parameter schemas.
5. ☐ **Binary Storage Driver:** Add an S3/filesystem streaming option for binary payloads > 10MB to avoid high worker memory spikes.
6. ☐ **Retention Cron:** Validate background execution cleanup based on `EXECUTION_RETENTION_DAYS`.